

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность ПО-13

Выполнил:
М.А. Романюк
студент группы ПО-13
Проверил:
А.А.Крощенко
доц. кафедры ИИТ,
14.05.2026

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Общее задание:

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику.

При реализации продумать типизацию полей и внешние ключи в таблицах;

2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);

3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;

4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);

5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпоинт;

Базу данные можно реализовать в любой СУБД (MySQL, PostgreSQL, SQLite и др.)

Задание:

Вариант 9. База данных Справочное бюро ж/д вокзала

Таблицы в БД:

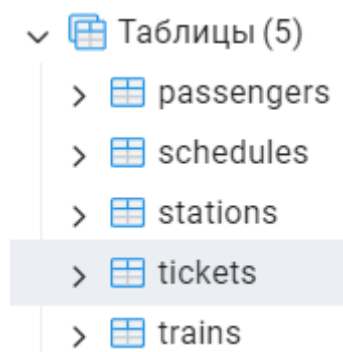
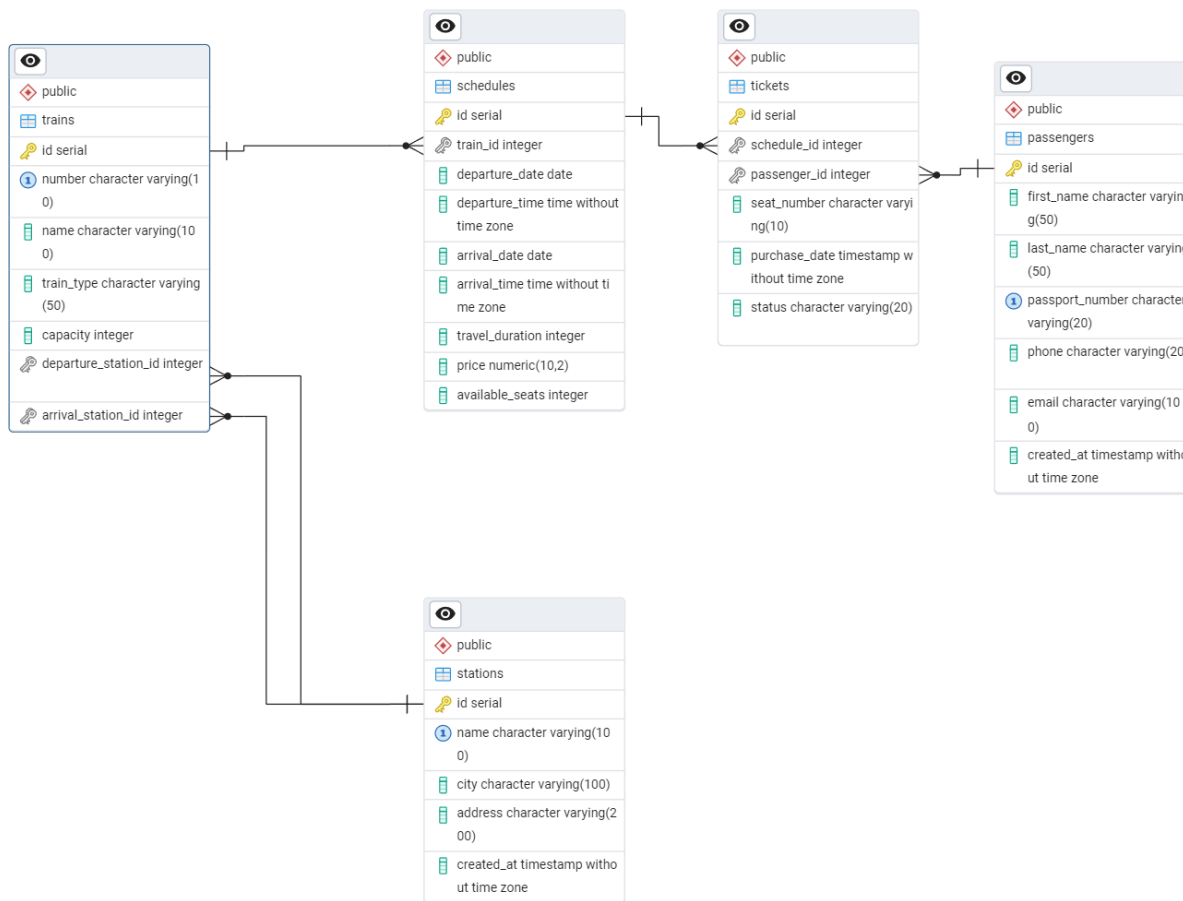


Схема БД в PostgreSQL:



Для запуска необходимо использовать python main.py затем перейти по ссылке <http://127.0.0.1:8000/docs>

Код программы:

crud.py

```

# -*- coding: utf-8 -*-
"""CRUD operations for database."""

from datetime import date

from sqlalchemy import and_
from sqlalchemy.orm import Session

import models
import schemas

def create_station(db: Session, station: schemas.StationCreate):
    """Create a new station."""
  
```

```

    db_station = models.Station(**station.model_dump())
    db.add(db_station)
    db.commit()
    db.refresh(db_station)
    return db_station

def get_stations(db: Session, skip: int = 0, limit: int = 100):
    """Get all stations."""
    return db.query(models.Station).offset(skip).limit(limit).all()

def get_station(db: Session, station_id: int):
    """Get station by ID."""
    return db.query(models.Station).filter(models.Station.id == station_id).first()

def update_station(db: Session, station_id: int, station_data: schemas.StationCreate):
    """Update station."""
    db_station = get_station(db, station_id)
    if db_station:
        for key, value in station_data.model_dump().items():
            setattr(db_station, key, value)
        db.commit()
        db.refresh(db_station)
    return db_station

def delete_station(db: Session, station_id: int):
    """Delete station."""
    db_station = get_station(db, station_id)
    if db_station:
        db.delete(db_station)
        db.commit()
        return True
    return False

def create_train(db: Session, train: schemas.TrainCreate):
    """Create a new train."""
    db_train = models.Train(**train.model_dump())
    db.add(db_train)
    db.commit()
    db.refresh(db_train)
    return db_train

def get_trains(db: Session, skip: int = 0, limit: int = 100):

```

```

    """Get all trains."""
    return db.query(models.Train).offset(skip).limit(limit).all()

def get_train(db: Session, train_id: int):
    """Get train by ID."""
    return db.query(models.Train).filter(models.Train.id == train_id).first()

def update_train(db: Session, train_id: int, train_data: schemas.TrainCreate):
    """Update train."""
    db_train = get_train(db, train_id)
    if db_train:
        for key, value in train_data.model_dump().items():
            setattr(db_train, key, value)
        db.commit()
        db.refresh(db_train)
    return db_train

def delete_train(db: Session, train_id: int):
    """Delete train."""
    db_train = get_train(db, train_id)
    if db_train:
        db.delete(db_train)
        db.commit()
        return True
    return False

def create_schedule(db: Session, schedule: schemas.ScheduleCreate):
    """Create a new schedule."""
    db_schedule = models.Schedule(**schedule.model_dump())
    db.add(db_schedule)
    db.commit()
    db.refresh(db_schedule)
    return db_schedule

def get_schedules(db: Session, skip: int = 0, limit: int = 100):
    """Get all schedules."""
    return db.query(models.Schedule).offset(skip).limit(limit).all()

def get_schedule(db: Session, schedule_id: int):
    """Get schedule by ID."""
    return db.query(models.Schedule).filter(models.Schedule.id == schedule_id).first()

```

```

def update_schedule(db: Session, schedule_id: int, schedule_data: schemas.ScheduleCreate):
    """Update schedule."""
    db_schedule = get_schedule(db, schedule_id)
    if db_schedule:
        for key, value in schedule_data.model_dump().items():
            setattr(db_schedule, key, value)
        db.commit()
        db.refresh(db_schedule)
    return db_schedule

def delete_schedule(db: Session, schedule_id: int):
    """Delete schedule."""
    db_schedule = get_schedule(db, schedule_id)
    if db_schedule:
        db.delete(db_schedule)
        db.commit()
        return True
    return False

def find_trains_by_station_and_date(db: Session, station_id: int, travel_date: date):
    """Найти поезда по станции и дате отправления."""
    results = (
        db.query(models.Schedule)
        .join(models.Train)
        .filter(and_(models.Train.departure_station_id == station_id,
models.Schedule.departure_date == travel_date))
        .all()
    )
    return results

def create_passenger(db: Session, passenger: schemas.PassengerCreate):
    """Create a new passenger."""
    db_passenger = models.Passenger(**passenger.model_dump())
    db.add(db_passenger)
    db.commit()
    db.refresh(db_passenger)
    return db_passenger

def get_passengers(db: Session, skip: int = 0, limit: int = 100):
    """Get all passengers."""
    return db.query(models.Passenger).offset(skip).limit(limit).all()

def get_passenger(db: Session, passenger_id: int):

```

```

    """Get passenger by ID."""
    return db.query(models.Passenger).filter(models.Passenger.id == passenger_id).first()

def update_passenger(db: Session, passenger_id: int, passenger_data: schemas.PassengerCreate):
    """Update passenger."""
    db_passenger = get_passenger(db, passenger_id)
    if db_passenger:
        for key, value in passenger_data.model_dump().items():
            setattr(db_passenger, key, value)
        db.commit()
        db.refresh(db_passenger)
    return db_passenger

def delete_passenger(db: Session, passenger_id: int):
    """Delete passenger."""
    db_passenger = get_passenger(db, passenger_id)
    if db_passenger:
        db.delete(db_passenger)
        db.commit()
        return True
    return False

def create_ticket(db: Session, ticket: schemas.TicketCreate):
    """Create a new ticket and reduce available seats."""
    schedule = db.query(models.Schedule).filter(models.Schedule.id ==
ticket.schedule_id).first()
    if schedule and schedule.available_seats > 0:
        schedule.available_seats -= 1
        db_ticket = models.Ticket(**ticket.model_dump())
        db.add(db_ticket)
        db.commit()
        db.refresh(db_ticket)
        return db_ticket
    return None

def get_tickets(db: Session, skip: int = 0, limit: int = 100):
    """Get all tickets."""
    return db.query(models.Ticket).offset(skip).limit(limit).all()

def get_ticket(db: Session, ticket_id: int):
    """Get ticket by ID."""
    return db.query(models.Ticket).filter(models.Ticket.id == ticket_id).first()

```

```

def update_ticket(db: Session, ticket_id: int, ticket_data: schemas.TicketCreate):
    """Update ticket."""
    db_ticket = get_ticket(db, ticket_id)
    if db_ticket:
        for key, value in ticket_data.model_dump().items():
            setattr(db_ticket, key, value)
        db.commit()
        db.refresh(db_ticket)
    return db_ticket

def delete_ticket(db: Session, ticket_id: int):
    """Delete ticket and restore available seats."""
    db_ticket = get_ticket(db, ticket_id)
    if db_ticket:
        schedule = db.query(models.Schedule).filter(models.Schedule.id ==
db_ticket.schedule_id).first()
        if schedule:
            schedule.available_seats += 1
        db.delete(db_ticket)
        db.commit()
        return True
    return False

```

database.py

```

# pylint: disable=invalid-name
"""Database connection and session management."""
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "postgresql://postgres:0000@localhost:5432/railway_bureau"

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    """Get database session."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```


main.py

```
# -*- coding: utf-8 -*-
"""FastAPI application for Railway Bureau."""

from datetime import date
from typing import List

from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session

import models
import schemas
import crud
from database import engine, get_db

models.Base.metadata.create_all(bind=engine)

app = FastAPI(title="Railway Bureau", description="Справочное бюро ж/д вокзала")

@app.post("/stations/", response_model=schemas.StationResponse)
def create_station(station: schemas.StationCreate, db: Session = Depends(get_db)):
    """Create a new station."""
    return crud.create_station(db, station)

@app.get("/stations/", response_model=List[schemas.StationResponse])
def get_stations(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    """Get all stations with pagination."""
    return crud.get_stations(db, skip, limit)

@app.get("/stations/{station_id}", response_model=schemas.StationResponse)
def get_station(station_id: int, db: Session = Depends(get_db)):
    """Get station by ID."""
    station = crud.get_station(db, station_id)
    if not station:
        raise HTTPException(404, "Station not found")
    return station

@app.put("/stations/{station_id}", response_model=schemas.StationResponse)
def update_station(station_id: int, station: schemas.StationCreate, db: Session = Depends(get_db)):
    """Update station by ID."""
    result = crud.update_station(db, station_id, station)
    if not result:
```

```

        raise HTTPException(404, "Station not found")
    return result

@app.delete("/stations/{station_id}")
def delete_station(station_id: int, db: Session = Depends(get_db)):
    """Delete station by ID."""
    if not crud.delete_station(db, station_id):
        raise HTTPException(404, "Station not found")
    return {"message": "Station deleted"}

@app.post("/trains/", response_model=schemas.TrainResponse)
def create_train(train: schemas.TrainCreate, db: Session = Depends(get_db)):
    """Create a new train."""
    return crud.create_train(db, train)

@app.get("/trains/", response_model=List[schemas.TrainResponse])
def get_trains(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    """Get all trains with pagination."""
    return crud.get_trains(db, skip, limit)

@app.get("/trains/{train_id}", response_model=schemas.TrainResponse)
def get_train(train_id: int, db: Session = Depends(get_db)):
    """Get train by ID."""
    train = crud.get_train(db, train_id)
    if not train:
        raise HTTPException(404, "Train not found")
    return train

@app.put("/trains/{train_id}", response_model=schemas.TrainResponse)
def update_train(train_id: int, train: schemas.TrainCreate, db: Session = Depends(get_db)):
    """Update train by ID."""
    result = crud.update_train(db, train_id, train)
    if not result:
        raise HTTPException(404, "Train not found")
    return result

@app.delete("/trains/{train_id}")
def delete_train(train_id: int, db: Session = Depends(get_db)):
    """Delete train by ID."""
    if not crud.delete_train(db, train_id):
        raise HTTPException(404, "Train not found")
    return {"message": "Train deleted"}

```

```

@app.post("/schedules/", response_model=schemas.ScheduleResponse)
def create_schedule(schedule: schemas.ScheduleCreate, db: Session = Depends(get_db)):
    """Create a new schedule."""
    return crud.create_schedule(db, schedule)


@app.get("/schedules/", response_model=List[schemas.ScheduleResponse])
def get_schedules(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    """Get all schedules with pagination."""
    return crud.get_schedules(db, skip, limit)


@app.get("/schedules/{schedule_id}", response_model=schemas.ScheduleResponse)
def get_schedule(schedule_id: int, db: Session = Depends(get_db)):
    """Get schedule by ID."""
    schedule = crud.get_schedule(db, schedule_id)
    if not schedule:
        raise HTTPException(404, "Schedule not found")
    return schedule


@app.put("/schedules/{schedule_id}", response_model=schemas.ScheduleResponse)
def update_schedule(schedule_id: int, schedule: schemas.ScheduleCreate, db: Session = Depends(get_db)):
    """Update schedule by ID."""
    result = crud.update_schedule(db, schedule_id, schedule)
    if not result:
        raise HTTPException(404, "Schedule not found")
    return result


@app.delete("/schedules/{schedule_id}")
def delete_schedule(schedule_id: int, db: Session = Depends(get_db)):
    """Delete schedule by ID."""
    if not crud.delete_schedule(db, schedule_id):
        raise HTTPException(404, "Schedule not found")
    return {"message": "Schedule deleted"}


@app.get("/schedules/search/by-station/")
def search_by_station(station_id: int, travel_date: date, db: Session = Depends(get_db)):
    """Search schedules by station ID and travel date."""
    return crud.find_trains_by_station_and_date(db, station_id, travel_date)


@app.post("/passengers/", response_model=schemas.PassengerResponse)
def create_passenger(passenger: schemas.PassengerCreate, db: Session = Depends(get_db)):

```

```

    """Create a new passenger."""
    return crud.create_passenger(db, passenger)

@app.get("/passengers/", response_model=List[schemas.PassengerResponse])
def get_passengers(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    """Get all passengers with pagination."""
    return crud.get_passengers(db, skip, limit)

@app.get("/passengers/{passenger_id}", response_model=schemas.PassengerResponse)
def get_passenger(passenger_id: int, db: Session = Depends(get_db)):
    """Get passenger by ID."""
    passenger = crud.get_passenger(db, passenger_id)
    if not passenger:
        raise HTTPException(404, "Passenger not found")
    return passenger

@app.put("/passengers/{passenger_id}", response_model=schemas.PassengerResponse)
def update_passenger(passenger_id: int, passenger: schemas.PassengerCreate, db: Session = Depends(get_db)):
    """Update passenger by ID."""
    result = crud.update_passenger(db, passenger_id, passenger)
    if not result:
        raise HTTPException(404, "Passenger not found")
    return result

@app.delete("/passengers/{passenger_id}")
def delete_passenger(passenger_id: int, db: Session = Depends(get_db)):
    """Delete passenger by ID."""
    if not crud.delete_passenger(db, passenger_id):
        raise HTTPException(404, "Passenger not found")
    return {"message": "Passenger deleted"}

@app.post("/tickets/", response_model=schemas.TicketResponse)
def create_ticket(ticket: schemas.TicketCreate, db: Session = Depends(get_db)):
    """Create a new ticket."""
    result = crud.create_ticket(db, ticket)
    if not result:
        raise HTTPException(400, "No available seats")
    return result

@app.get("/tickets/", response_model=List[schemas.TicketResponse])
def get_tickets(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):

```

```

    """Get all tickets with pagination."""
    return crud.get_tickets(db, skip, limit)

@app.get("/tickets/{ticket_id}", response_model=schemas.TicketResponse)
def get_ticket(ticket_id: int, db: Session = Depends(get_db)):
    """Get ticket by ID."""
    ticket = crud.get_ticket(db, ticket_id)
    if not ticket:
        raise HTTPException(404, "Ticket not found")
    return ticket

@app.put("/tickets/{ticket_id}", response_model=schemas.TicketResponse)
def update_ticket(ticket_id: int, ticket: schemas.TicketCreate, db: Session =
Depends(get_db)):
    """Update ticket by ID."""
    result = crud.update_ticket(db, ticket_id, ticket)
    if not result:
        raise HTTPException(404, "Ticket not found")
    return result

@app.delete("/tickets/{ticket_id}")
def delete_ticket(ticket_id: int, db: Session = Depends(get_db)):
    """Delete ticket by ID."""
    if not crud.delete_ticket(db, ticket_id):
        raise HTTPException(404, "Ticket not found")
    return {"message": "Ticket deleted"}

if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="127.0.0.1", port=8000)

```

models.py

```

# -*- coding: utf-8 -*-
"""Database models for railway bureau."""
from datetime import datetime
from sqlalchemy import Column, Integer, String, Date, Time, ForeignKey, Numeric,
CheckConstraint, DateTime
from sqlalchemy.orm import relationship
from database import Base

```

```

class Station(Base):
    """Station model - таблица станций."""
    __tablename__ = 'stations'

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False, unique=True)
    city = Column(String(100), nullable=False)
    address = Column(String(200))
    created_at = Column(DateTime, default=datetime.now)

    # Связи
    departures = relationship('Train', foreign_keys='Train.departure_station_id',
back_populates='departure_station')
    arrivals = relationship('Train', foreign_keys='Train.arrival_station_id',
back_populates='arrival_station')

class Train(Base):
    """Train model - таблица поездов."""
    __tablename__ = 'trains'

    id = Column(Integer, primary_key=True, index=True)
    number = Column(String(10), nullable=False, unique=True)
    name = Column(String(100))
    train_type = Column(String(50), nullable=False)
    capacity = Column(Integer, nullable=False)

    departure_station_id = Column(Integer, ForeignKey('stations.id'), nullable=False)
    arrival_station_id = Column(Integer, ForeignKey('stations.id'), nullable=False)

    # Связи
    departure_station = relationship('Station', foreign_keys=[departure_station_id],
back_populates='departures')
    arrival_station = relationship('Station', foreign_keys=[arrival_station_id],
back_populates='arrivals')
    schedules = relationship('Schedule', back_populates='train')

    __table_args__ = (
        CheckConstraint('capacity > 0', name='check_capacity_positive'),
    )

class Schedule(Base):
    """Schedule model - таблица расписания."""
    __tablename__ = 'schedules'

    id = Column(Integer, primary_key=True, index=True)
    train_id = Column(Integer, ForeignKey('trains.id'), nullable=False)

```

```

departure_date = Column(Date, nullable=False)
departure_time = Column(Time, nullable=False)
arrival_date = Column(Date, nullable=False)
arrival_time = Column(Time, nullable=False)
travel_duration = Column(Integer)
price = Column(Numeric(10, 2), nullable=False)
available_seats = Column(Integer, nullable=False)

train = relationship('Train', back_populates='schedules')
tickets = relationship('Ticket', back_populates='schedule')

__table_args__ = (
    CheckConstraint('available_seats >= 0', name='check_seats_positive'),
    CheckConstraint('price > 0', name='check_price_positive'),
)

```

```

class Passenger(Base):
    """Passenger model - таблица пассажиров."""
    __tablename__ = 'passengers'

    id = Column(Integer, primary_key=True, index=True)
    first_name = Column(String(50), nullable=False)
    last_name = Column(String(50), nullable=False)
    passport_number = Column(String(20), nullable=False, unique=True)
    phone = Column(String(20))
    email = Column(String(100))
    created_at = Column(DateTime, default=datetime.now)

    tickets = relationship('Ticket', back_populates='passenger')

```

```

class Ticket(Base):
    """Ticket model - таблица билетов."""
    __tablename__ = 'tickets'

    id = Column(Integer, primary_key=True, index=True)
    schedule_id = Column(Integer, ForeignKey('schedules.id'), nullable=False)
    passenger_id = Column(Integer, ForeignKey('passengers.id'), nullable=False)
    seat_number = Column(String(10), nullable=False)
    purchase_date = Column(DateTime, default=datetime.now)
    status = Column(String(20), default='active') # active, cancelled, used

    schedule = relationship('Schedule', back_populates='tickets')
    passenger = relationship('Passenger', back_populates='tickets')

```

requirements.txt

```
fastapi==0.104.1
uvicorn==0.24.0
sqlalchemy==2.0.23
psycopg2-binary==2.9.9
pydantic==2.5.0
requests==2.31.0
```

schemas.py

```
"""Pydantic schemas for request/response validation."""
from datetime import date, time, datetime
from decimal import Decimal
from typing import Optional

from pydantic import BaseModel

class StationBase(BaseModel):
    """Base schema for station."""
    name: str
    city: str
    address: Optional[str] = None

class StationCreate(StationBase):
    """Schema for creating a new station."""

class StationResponse(StationBase):
    """Schema for station response."""
    id: int
    created_at: datetime

    class Config:
        """Pydantic config."""
        from_attributes = True

class TrainBase(BaseModel):
    """Base schema for train."""
    number: str
    name: Optional[str] = None
    train_type: str
    capacity: int
    departure_station_id: int
    arrival_station_id: int
```



```

class TrainCreate(TrainBase):
    """Schema for creating a new train."""

class TrainResponse(TrainBase):
    """Schema for train response."""
    id: int

    class Config:
        """Pydantic config."""
        from_attributes = True

class ScheduleBase(BaseModel):
    """Base schema for schedule."""
    train_id: int
    departure_date: date
    departure_time: time
    arrival_date: date
    arrival_time: time
    travel_duration: Optional[int] = None
    price: Decimal
    available_seats: int

class ScheduleCreate(ScheduleBase):
    """Schema for creating a new schedule."""

class ScheduleResponse(ScheduleBase):
    """Schema for schedule response."""
    id: int

    class Config:
        """Pydantic config."""
        from_attributes = True

class PassengerBase(BaseModel):
    """Base schema for passenger."""
    first_name: str
    last_name: str
    passport_number: str
    phone: Optional[str] = None
    email: Optional[str] = None

class PassengerCreate(PassengerBase):
    """Schema for creating a new passenger."""

```

```

class PassengerResponse(PassengerBase):
    """Schema for passenger response."""
    id: int
    created_at: datetime

    class Config:
        """Pydantic config."""
        from_attributes = True


class TicketBase(BaseModel):
    """Base schema for ticket."""
    schedule_id: int
    passenger_id: int
    seat_number: str
    status: Optional[str] = "active"


class TicketCreate(TicketBase):
    """Schema for creating a new ticket."""


class TicketResponse(TicketBase):
    """Schema for ticket response."""
    id: int
    purchase_date: datetime

    class Config:
        """Pydantic config."""
        from_attributes = True

```

Рисунки с результатами работы программы

Railway Bureau

0.1.0 OAS 3.1

Агрегатор.json

Справочное бюро ж/д вокзала

default

POST	/stations/	Create Station
GET	/stations/	Get Stations
GET	/stations/{station_id}	Get Station
PUT	/stations/{station_id}	Update Station
DELETE	/stations/{station_id}	Delete Station
POST	/trains/	Create Train
GET	/trains/	Get Trains
GET	/trains/{train_id}	Get Train
PUT	/trains/{train_id}	Update Train
DELETE	/trains/{train_id}	Delete Train
POST	/schedules/	Create Schedule
GET	/schedules/	Get Schedules
GET	/schedules/{schedule_id}	Get Schedule
PUT	/schedules/{schedule_id}	Update Schedule
DELETE	/schedules/{schedule_id}	Delete Schedule
GET	/schedules/search/by-station/	Search By Station
POST	/passengers/	Create Passenger
GET	/passengers/	Get Passengers
GET	/passengers/{passenger_id}	Get Passenger
PUT	/passengers/{passenger_id}	Update Passenger
DELETE	/passengers/{passenger_id}	Delete Passenger

GET stations:

Code	Details
200	Response body <pre>[{ "name": "Brest-central", "city": "Brest", "address": "pl. Privokzalnaya, 1", "id": 1, "created_at": "2026-04-17T10:01:38.351530" }, { "name": "Gabinka", "city": "Gabinka", "address": "ul. Central, 12", "id": 2, "created_at": "2026-04-17T10:02:45.758485" }, { "name": "Москва-Пассажирская", "city": "Москва", "address": "Комсомольская пл., 3", "id": 3, "created_at": "2026-05-14T18:37:34.090202" }, { "name": "Санкт-Петербург-Главный", "city": "Санкт-Петербург", "address": "Невский пр., 85", "id": 4, "created_at": "2026-05-14T18:37:44.033556" }, { "name": "Витебск", "city": "Витебск", "address": "Невский пр., 85", "id": 5, "created_at": "2026-05-14T18:44:41.837565" }]</pre>

	id [PK] integer	name character varying (100)	city character varying (100)	address character varying (200)	created_at timestamp without time zone
1	1	Brest-central	Brest	pl. Privokzalnaya, 1	2026-04-17 10:01:38.35153
2	2	Gabinka	Gabinka	ul. Central, 12	2026-04-17 10:02:45.758485
3	3	Москва-Пассажирская	Москва	Комсомольская пл., 3	2026-05-14 18:37:34.090202
4	4	Санкт-Петербург-Главный	Санкт-Петербург	Невский пр., 85	2026-05-14 18:37:44.033556
5	5	Витебск	Витебск	Невский пр., 85	2026-05-14 18:44:41.837565

POST stations:

Code	Details
200	Response body <pre>{ "name": "Grodno", "city": "Grodno", "address": "ul. Gogolya 16", "id": 6, "created_at": "2026-05-14T19:24:48.204158" }</pre>

6	6	Grodno	Grodno	ul. Gogolya 16	2026-05-14 19:24:48.204158
---	---	--------	--------	----------------	----------------------------

DELETE stations (6)

Code	Details
200	Response body <pre>{ "message": "Station deleted" }</pre>

Вывод: приобрел практические навыки разработки API и баз данных